PilotFish Route to Route Transport – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

This transport **hands the message to another route** by triggering a **Programmable (Trigger)** listener by name — like calling a subroutine. You can fire-and-forget, or wait up to a timeout for that other route to finish and optionally forward its response to a second listener.

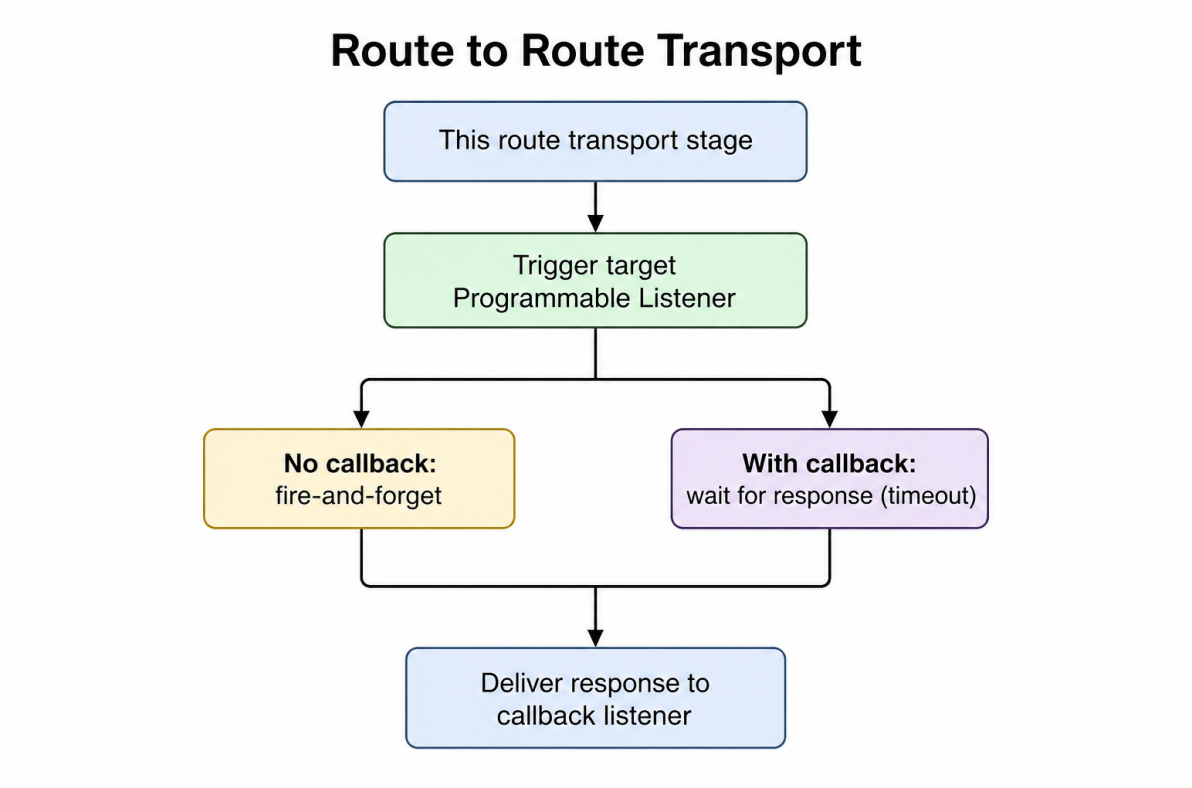

Figure 1: At a glance (plain English)

- **Targets** a named Programmable (Trigger) listener (`ServiceName`).
- **Async mode** — trigger and continue when no Response Listener is configured.
- **Sync mode** — wait for target route completion (`Timeout` seconds), replace body with response.

- **Callback** — optionally invoke a second listener with the response stream.

Purpose

This document is a **deep dive** into the **Route to Route** transport (`EIPTransport`) as shipped in **`eip.war.hs.26R1.11`**: programmatic listener triggering, synchronous wait, response caching, and callback delivery.

Provenance	Value
WAR	`eip.war.hs.26R1.11`
Module JAR	`xcs-modules-core-1.0.3.jar`
Decompiler	CFR 0.152
Implementation class	`com.pilotfish.eip.modules.internal.EIPTran`
Decompiled path	`war-pipeline/decompiled/eip-26R1.11/.../EI`
Transport type string	`Route to Route`
Description	`Sends data to a specific programmatically-invoked listener.`
Help ID	`console.transport.routetoroute`
Category	`PILOT_FISH`

---

1. Conceptual model

Route to Route connects routes **by listener name**, not by route ID. The target must be a **Programmable (Trigger)** listener (or compatible custom listener) that accepts programmatic invocation.

```
Route A (transport stage)
     |
     +-- CallbackListenerName empty?
     |       yes -> TriggerableListenerUtils.trigger(ServiceName)  [async]
     |
     +-- CallbackListenerName set?
             -> triggerAndWait(ServiceName, timeout)
             -> cache response stream onto original data
             -> trigger(CallbackListenerName, response, newData)
```

Implements `AbstractReferencedListeners` for rename safety when listener names change in the model.

---

2. High-level behavior (`executeTransport`)

1. Read **Target Listener** (`ServiceName`) and **Response Listener** (`CallbackListenerName`).
2. **If Response Listener is empty**
   - `TriggerableListenerUtils.trigger(ServiceName, data.getDataStream(), data, this)` — asynchronous handoff.

3. **If Response Listener is configured**
   - `TriggerableListenerUtils.triggerAndWait(ServiceName, data, Timeout, this)`
   - `null` result → `EIPException("Transaction timed out.")`
   - Copy response: `DataUtil.copyAndClose(response, cache)`; `data.setDataStream(cache.getInputS`
   - `TriggerableListenerUtils.trigger(CallbackListenerName, response, newData, this)` using returned `TransactionData`
4. Exceptions wrapped as `EIPException("Unable to copy EIPTransport response to cache contents", cause)`.

Standard transport wrapper sets transaction complete on success.

---

3. Configuration reference

3.1 Module-specific options

UI label	Tag	Type	Default	Tab	Required	Nuances
Target Listener	`ServiceName`	Listener	`""`	Basic	Yes	Filter: `Programmable (Trigger)`
Response Listener	`CallbackListenerName`	Listener	`""`	Basic	No	Enables sync + callback path when set
Timeout (sec)	`Timeout`	Integer	`60`	Basic	No	Min 5, max `Integer.MAX_VALUE`; wait only in callback mode

Timeout nuance: Tooltip states timeout does **not abort** the invoked route — it only stops waiting and fails this transport.

3.2 Listener reference maintenance

Method	Behavior
`referencedProgramListenerNameIsUsed(name)`	True if name equals `ServiceName` or `CallbackListenerName`
`replaceProgramListenerName(old, new)`	Updates whichever properties match

3.3 Inherited AbstractTransport

Standard `sendData` → `checkRequiredData` → `executeTransport` → complete/error states.

---

4. Transaction attributes

No attributes are registered by `EIPTransport` itself. Inherited attributes from source/target routes pass through `TransactionData` clones returned by `triggerAndWait`.

---

5. Worked examples

5.1 Fire-and-forget subroutine

- **Target Listener:** `ProcessClaimTrigger`
- **Response Listener:** *(empty)*

Each transaction triggers `ProcessClaimTrigger` with the current stream; Route A completes without waiting.

5.2 Request/response chain

- **Target Listener:** `PricingEngineTrigger`
- **Response Listener:** `PricingResponseHandler`
- **Timeout:** `120`

Route A waits up to 120 seconds for the pricing route, replaces its body with the pricing response, then invokes `PricingResponseHandler` with the **`newData`** instance from the wait call.

---

6. Known quirks and caveats

1. **Timeout does not cancel** remote route work — only fails the waiter.
2. **Callback requires Response Listener** — sync wait path is only entered when callback name is non-empty (not merely when you want a synchronous response on the same transaction).
3. **`InputStream.available()`** used for cache sizing hint when copying response.
4. **Listener type filter** — UI restricts picker to Programmable (Trigger); mismatched types fail at runtime.
5. **Stream ownership** — triggering passes current stream; caller route should not assume stream remains open.

---

7. Operational recommendations

Goal	Guidance
Subroute reuse	Centralize logic behind one trigger listener per concern
Latency bounds	Set **Timeout** above p99 subroute duration; monitor timeout errors

Goal	Guidance
Response fan-out	Use **Response Listener** when a third route must react to subroute output
Refactoring	Rely on referenced-listener rename hooks when renaming trigger listeners

---

8. Source index (this WAR)

Topic	Path under `war-pipeline/decompiled/eip-26R1.11/`
Route to Route transport	`com/pilotfish/eip/modules/internal/EIPTransport.java`
Trigger utilities	`com/pilotfish/eip/TriggerableListenerUtils.java`
Abstract transport	`com/pilotfish/eip/extend/AbstractTransport.java`
Module JAR	`war-pipeline/jars/eip-26R1.11/xcs-modules-core-1.0.3.jar`

---

9. Pipeline provenance

```
PilotFish WARs/eip.war.hs.26R1.11
  -> xcs-modules-core-1.0.3.jar
  -> CFR decompile
  -> Documents/Transports/26R1.11/PilotFish-Route-to-Route-Transport-Reference-26R1.11.(md|pdf)
```

Deep-dive documentation from WAR decompile – Route to Route Transport – 26R1.11 – August 2026.